

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Automação de Testes Web

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Testes de Automação Web

Automação Web
Aula 3



Estrutura do curso

Conteúdo:

Aula 1:

- Fundamentos da Automação de Testes
- Diferenças entre testes manuais e automatizados
- Estrutura de um Script de Teste
- Ferramentas

Aula 2:

- Identificação de Elementos
- Elementos dinâmicos e boas práticas
- Introdução ao padrão page object

Aula 3:

- Organização com page object
- Comandos, esperas e interações básicas
- Comandos, esperas e validações

Aula 4:

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Aula 5:

- Criação de fluxo de teste completo
- Prática guiada
- Bonus: CI/CD + Paralelização

Intervalo: 15 à 20 minutos

Material das aulas: NEO

Avaliação Final: 5 questões objetivas

O que veremos hoje?

- Organização com page object
- Comandos, esperas e interações básicas
- Comandos, esperas e validações

Organização com page object

Estrutura de Pastas

CSS

 Copy  Edit

```
src/  
├── pages/  
│   ├── LoginPage.java  
│   └── DashboardPage.java  
├── tests/  
│   └── LoginTest.java  
└── utils/  
    └── DriverFactory.java
```

Organização com page object

LoginPage.java

```
java Copy Edit

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
    private WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    private By campoUsuario = By.cssSelector("input[name='username']");
    private By campoSenha = By.xpath("//input[contains(@placeholder,'senha')]");
    private By botaoEntrar = By.cssSelector("button.btn-submit");

    public void preencherUsuario(String usuario) {
        driver.findElement(campoUsuario).sendKeys(usuario);
    }

    public void preencherSenha(String senha) {
        driver.findElement(campoSenha).sendKeys(senha);
    }

    public void clicarEntrar() {
        driver.findElement(botaoEntrar).click();
    }

    public void loginComo(String usuario, String senha) {
        preencherUsuario(usuario);
        preencherSenha(senha);
        clicarEntrar();
    }
}
```

DashboardPage.java

```
java Copy Edit

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class DashboardPage {
    private WebDriver driver;

    public DashboardPage(WebDriver driver) {
        this.driver = driver;
    }

    private By mensagemBoasVindas = By.cssSelector("div.welcome-message");
    private By menuUsuario = By.cssSelector("nav .user-menu");

    public String obterMensagemBoasVindas() {
        return driver.findElement(mensagemBoasVindas).getText();
    }

    public boolean menuUsuarioVisivel() {
        return driver.findElement(menuUsuario).isDisplayed();
    }
}
```

Organização com page object

```
public class LoginTest {  
    private WebDriver driver;  
    private LoginPage loginPage;  
    private DashboardPage dashboardPage;  
  
    @Before  
    public void setUp() {  
        driver = DriverFactory.createDriver();  
        driver.get("https://exemplo.com/login");  
  
        loginPage = new LoginPage(driver);  
        dashboardPage = new DashboardPage(driver);  
    }  
  
    @Test  
    public void testLoginEValidacaoDashboard() {  
        loginPage.loginComo("meu_usuario", "minha_senha");  
  
        // Validações na Dashboard  
        assertTrue(dashboardPage.menuUsuarioVisivel());  
        assertEquals("Bem-vindo, Thiago!", dashboardPage.obterMensagemBoasVindas());  
    }  
  
    @After  
    public void tearDown() {  
        driver.quit();  
    }  
}
```

Organização com page object

O que muda com o uso de Page Factory?

Em vez de escrever:

java

```
private By campoUsuario = By.cssSelector("input[name='username']");  
driver.findElement(campoUsuario).sendKeys("usuario");
```

Com Page Factory usamos:

java

```
@FindBy(css = "input[name='username']")  
private WebElement campoUsuario;  
campoUsuario.sendKeys("usuario");
```

Organização com page object

✓ LoginPage.java com Page Factory

```
java

package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    private WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this); // inicializa os elementos
    }

    @FindBy(css = "input[name='username']")
    private WebElement campoUsuario;

    @FindBy(xpath = "//input[contains(@placeholder,'senha')]")
    private WebElement campoSenha;

    @FindBy(css = "button.btn-submit")
    private WebElement botaoEntrar;

    public void preencherUsuario(String usuario) {
        campoUsuario.sendKeys(usuario);
    }

    public void preencherSenha(String senha) {
        campoSenha.sendKeys(senha);
    }

    public void clicarEntrar() {
        botaoEntrar.click();
    }

    public void loginComo(String usuario, String senha) {
        preencherUsuario(usuario);
        preencherSenha(senha);
        clicarEntrar();
    }
}
```

Organização com page object

Quadro comparativo: com e sem Page Factory

Critério	Sem Page Factory	Com Page Factory
Mapeamento de elementos	Manual (com <code>By</code>)	Automático via <code>@FindBy</code>
Inicialização	Sempre usa <code>driver.findElement(...)</code>	Apenas uma vez com <code>PageFactory.initElements()</code>
Legibilidade	Médio: misto de código e localizadores	Alta: mais próximo da leitura natural
Boilerplate (código repetido)	Mais extenso: muitos <code>By</code> + <code>findElement()</code>	Reduzido: apenas os <code>@FindBy</code>
Manutenção	Centralizada, mas mais código para atualizar	Centralizada e mais limpa
Organização	Funcional, porém mais manual	Padrão de mercado mais moderno e limpo
Validação em tempo de compilação	Não valida se o elemento existe	Também não, mas torna mais legível o erro

Organização com page object

Quando usar Page Factory?

Ideal quando:

- Você quer um código mais limpo e direto.
- O número de elementos é grande.
- Você já usa boas práticas como Page Object.

Atenção:

- Não funciona bem com elementos carregados dinamicamente com AJAX, sem `waits`.
- Combine com `WebDriverWait` ou bibliotecas como `Awaitility` para robustez.

Waits??

Como assim ??



Comandos, esperas e interações básicas



Tipos de Wait no Selenium

Tipo de Wait	Controlado por quem?	Espera por quê?	Quando usar?
Implicit Wait	Selenium (global)	Qualquer <code>findElement()</code>	Simples, para elementos estáticos
Explicit Wait	Você (condições específicas)	Uma condição (ex: visibilidade)	Carregamento dinâmico, AJAX
Fluent Wait	Você (com polling)	Parecido com o <code>explicit</code> , mas mais flexível	Casos avançados, polling customizado

Comandos, esperas e interações básicas

1. 🧑 Implicit Wait (espera implícita)

Define um tempo padrão de espera para todos os `findElement`.

java

📄 Copy ✎ Edit

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

◆ Se o elemento não for encontrado imediatamente, o Selenium vai tentar por até 10 segundos antes de lançar exceção.

◆ Aplica-se a todos os elementos.

📌 Limitações:

- Não serve para condições como "elemento está visível" ou "clicável".
- Pode causar lentidão em testes grandes.

Comandos, esperas e interações básicas

2. 🔍 Explicit Wait (espera explícita)

Você espera por uma condição específica antes de seguir.

```
java                                                                    Copy Edit

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement botao = wait.until(ExpectedConditions.elementToBeClickable(By.id("enviar")))
botao.click();
```

📌 Principais condições (via `ExpectedConditions`):

Condição	Exemplo	📄
<code>visibilityOfElementLocated</code>	Quando o elemento está visível	
<code>elementToBeClickable</code>	Quando o botão pode ser clicado	
<code>presenceOfElementLocated</code>	Quando o elemento existe no DOM	
<code>invisibilityOfElementLocated</code>	Quando o carregamento desaparece	
<code>textToBePresentInElement</code>	Quando um texto aparece em um elemento	

💡 Recomendado para qualquer elemento que carregue dinamicamente.

Comandos, esperas e interações básicas

3. Fluent Wait

É uma variação do `ExplicitWait` com:

- Tempo máximo
- Intervalos de verificação (polling)
- Exceções ignoradas

java

 Copy  Edit

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(15))
    .pollingEvery(Duration.ofSeconds(2))
    .ignoring(NoSuchElementException.class);

WebElement campo = wait.until(driver -> driver.findElement(By.id("email")));
```

◆ Útil para sistemas instáveis, como SPAs ou redes lentas.

Comandos, esperas e interações básicas

Boas práticas

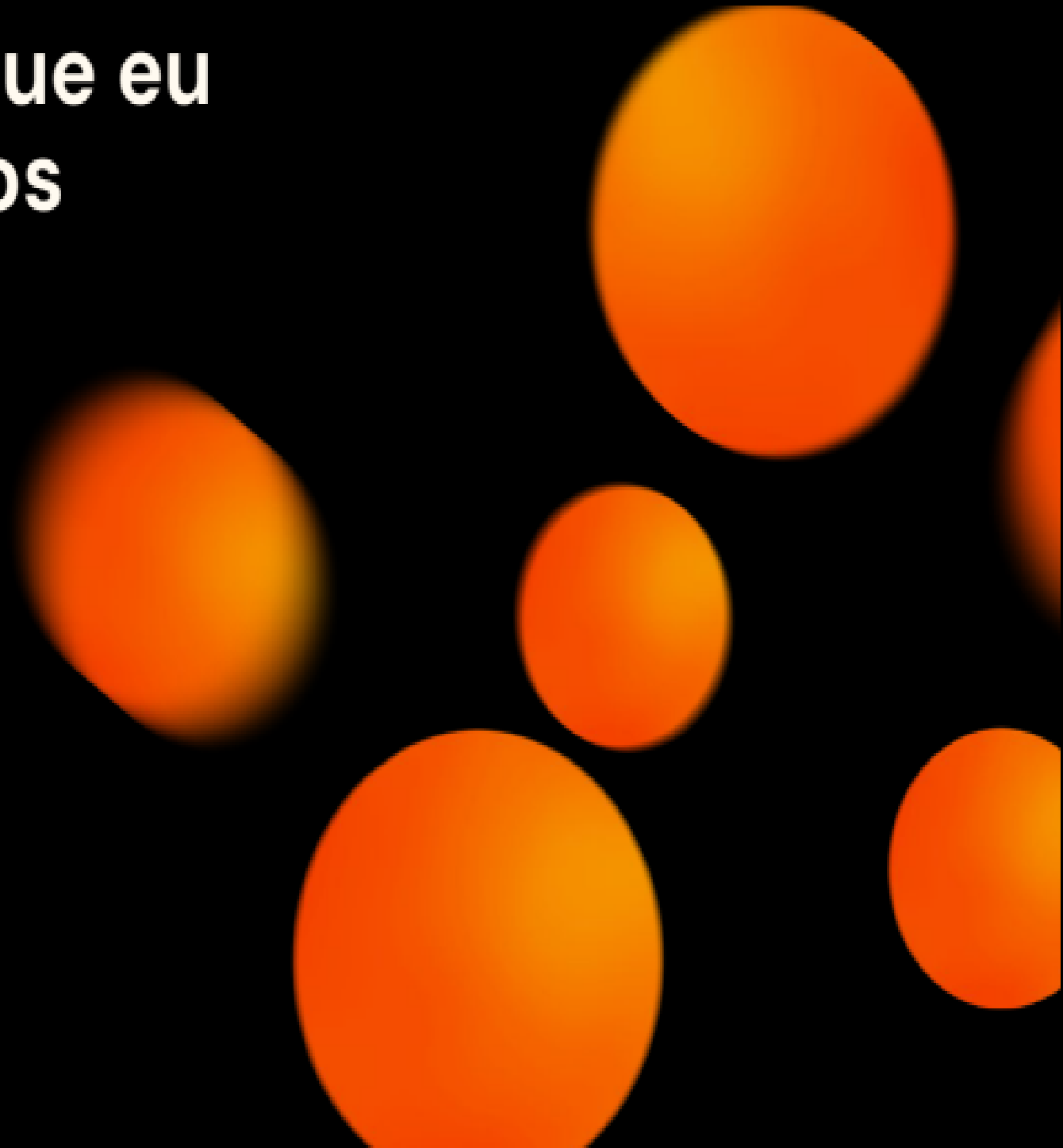
Boa prática	Explicação
Use <code>explicit wait</code> para elementos específicos	Evita lentidão e torna o teste mais previsível
Evite misturar <code>implicit</code> + <code>explicit</code>	Pode causar conflitos e falhas
Use <code>ExpectedConditions</code> apropriadas	Como <code>elementToBeClickable</code> , não apenas <code>presenceOfElementLocated</code>
Nunca use <code>Thread.sleep()</code> em produção	É uma espera fixa, sem inteligência e prejudica a performance
Combine PageFactory + Waits	Para tratar elementos dinâmicos de forma robusta

INTERVALO!



20 min.

**Beleza, mas o que eu
posso fazer com os
elementos ?**



Comandos, esperas e interações básicas

✓ Tabela 1 — Interações Básicas com WebElements (Selenium)

Essas são as ações diretas que você pode executar com elementos localizados via `findElement`.

Ação	Comando (Java - Selenium)	Descrição
Clicar	<code>element.click();</code>	Clica em botões, links, checkboxes etc.
Digitar texto	<code>element.sendKeys("texto");</code>	Preenche campos de input, textarea etc.
Limpar campo	<code>element.clear();</code>	Apaga o conteúdo de um campo de texto.
Obter texto do elemento	<code>element.getText();</code>	Retorna o texto visível do elemento.
Obter valor de atributo	<code>element.getAttribute("name");</code>	Retorna o valor de um atributo (<code>value</code> , <code>href</code> , etc.).
Verificar se está visível	<code>element.isDisplayed();</code>	Verifica se o elemento está visível na tela.

Comandos, esperas e interações básicas

Verificar se está ativo	<code>element.isEnabled();</code>	Verifica se está habilitado para interação.
Verificar se está selecionado	<code>element.isSelected();</code>	Para checkboxes ou radio buttons.
Selecionar item por texto	<code>new Select(element).selectByVisibleText("Opção");</code>	Para elementos <code><select></code> (dropdown).
Selecionar por valor	<code>new Select(element).selectByValue("1");</code>	Seleciona item com base no atributo <code>value</code> .
Selecionar por índice	<code>new Select(element).selectByIndex(0) ;</code>	Seleciona pela posição na lista de opções (0 é o primeiro).

Comandos, esperas e interações básicas

Tabela 2 — Ações Avançadas com a Classe `Actions`

Essas interações simulam ações reais do usuário com mouse e teclado.

Ação	Comando (Selenium + <code>Actions</code>)	Descrição
Mover mouse (hover)	<pre>new Actions(driver).moveToElement(el)perform();</pre>	Simula o mouse passando sobre o elemento.
Clique com botão direito	<pre>new Actions(driver).contextClick(el) .perform();</pre>	Abre o menu de contexto (clique direito).
Clique com o mouse	<pre>new Actions(driver).click(el).perfor m();</pre>	Clica usando a classe <code>Actions</code> .
Duplo clique	<pre>new Actions(driver).doubleClick(el). perform();</pre>	Simula um duplo clique.

Comandos, esperas e interações básicas

Clique e segure	<pre>new Actions(driver).clickAndHold(el) .perform();</pre>	Pressiona e mantém o clique do mouse.
Soltar (release)	<pre>new Actions(driver).release().perform();</pre>	Solta o botão do mouse (usado com <code>clickAndHold</code>).
Arrastar e soltar	<pre>new Actions(driver).dragAndDrop(src, tgt).perform();</pre>	Movimenta um elemento de uma posição a outra.
Enviar sequência de teclas	<pre>new Actions(driver).sendKeys("abc").perform();</pre>	Digita uma sequência no elemento ativo.
Pressionar tecla modificadora	<pre>keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();</pre>	Simula atalhos como Ctrl + A, Shift + Tab etc.

✓ Exemplo: moveToElement() + Explicit Wait + click()

```
public class HoverClickExample {  
    public static void main(String[] args) {  
  
        // Setup  
        WebDriver driver = new ChromeDriver();  
        driver.manage().window().maximize();  
        driver.get("https://sua-url-de-teste.com");  
  
        // Elementos principais  
        WebElement menuProdutos = driver.findElement(By.id("menu-produtos"));  
  
        // Hover para exibir submenu  
        Actions actions = new Actions(driver);  
        actions.moveToElement(menuProdutos).perform();  
  
        // Espere explícita para submenu aparecer  
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
        WebElement submenuNotebook = wait.until(  
            ExpectedConditions.visibilityOfElementLocated(By.id("submenu-notebook"))  
        );  
  
        // Clique no submenu  
        submenuNotebook.click();  
  
        // Fechamento do navegador  
        driver.quit();  
    }  
}
```

Comandos, esperas e validações

✓ Tabela de Exemplos — `assertEquals` e `assertTrue`

Tipo de Validação	Método	Exemplo de Uso Java (JUnit/TestNG)	Explicação
Texto exato	<code>assertEquals</code>	<pre>Assert.assertEquals("Bem-vindo", element.getText());</pre>	Verifica se o texto exibido é exatamente o esperado
Título da página	<code>assertEquals</code>	<pre>Assert.assertEquals("Página Inicial", driver.getTitle());</pre>	Compara o título atual da aba com o esperado
URL atual	<code>assertEquals</code>	<pre>Assert.assertEquals("https://site.com/home", driver.getCurrentUrl());</pre>	Garante que a navegação levou à URL correta
Atributo de um campo	<code>assertEquals</code>	<pre>Assert.assertEquals("admin", input.getAttribute("value"));</pre>	Valida o valor preenchido ou carregado em um campo
Quantidade de itens em lista	<code>assertEquals</code>	<pre>Assert.assertEquals(3, driver.findElements(By.className("item")).size());</pre>	Compara o número de elementos visíveis

Comandos, esperas e validações

✓ Tabela de Exemplos — `assertEquals` e `assertTrue`

Tipo de Validação	Método	Exemplo de Uso Java (JUnit/TestNG)	Explicação
Elemento visível	<code>assertTrue</code>	<code>Assert.assertTrue(element.isDisplayed());</code>	Confirma que o elemento está visível para o usuário
Elemento habilitado	<code>assertTrue</code>	<code>Assert.assertTrue(button.isEnabled());</code>	Verifica se é possível clicar/interagir com o botão
Checkbox selecionado	<code>assertTrue</code>	<code>Assert.assertTrue(checkbox.isSelected());</code>	Verifica se o checkbox foi marcado
Condição booleana customizada	<code>assertTrue</code>	<code>Assert.assertTrue(lista.size() > 0);</code>	Valida que a lista contém pelo menos um item
Visibilidade com wait	<code>assertTrue</code>	<code>Assert.assertTrue(wait.until(ExpectedConditions.visibilityOf(element)) != null);</code>	Confirma que o elemento se tornou visível com espera explícita

Comandos, esperas e validações

```
@Test
public void testLoginComSucesso() {
    // Preencher usuário e senha
    WebElement campoUsuario = driver.findElement(By.id("username"));
    campoUsuario.sendKeys("meu_usuario");

    WebElement campoSenha = driver.findElement(By.id("password"));
    campoSenha.sendKeys("minha_senha");

    // Clicar no botão Entrar
    WebElement botaoEntrar = driver.findElement(By.id("loginButton"));
    botaoEntrar.click();

    // Validar URL mudou para dashboard
    String urlEsperada = "https://exemplo.com/dashboard";
    Assert.assertEquals(urlEsperada, driver.getCurrentUrl());

    // Validar mensagem de boas-vindas
    WebElement mensagemBoasVindas = driver.findElement(By.id("welcomeMessage"));
    Assert.assertTrue(mensagemBoasVindas.isDisplayed());
    Assert.assertEquals("Bem-vindo, meu_usuario!", mensagemBoasVindas.getText());
}
```

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados